What you have been given, and what you must write

Read this before anything else. It is the map.

This repository is **deliberately incomplete**. Roughly half of it is supplied so you are not retyping lab code, and half is missing so that the assessment measures something. Every missing file has a skeleton in its place carrying the requirement, the traps, and how to check your answer — but not the answer.

The three kinds of file

	Meaning
GIVEN	Complete. Use it. You may change it if you can justify the change, and you must say so in your report if you do.
WORKED EXAMPLE	Complete, heavily commented, and the pattern for one or more files you must write yourself. Read it closely first.
TO WRITE	A comment-only skeleton. Replace it entirely. Nothing will deploy until you do.

The whole repository, classified

```
GIVEN
  app/catalog/app.py                 all three services, instrumented
  app/orders/app.py                  (Lab 9.3 + Lab 9.4 applied to it)
  app/frontend/app.py  templates/
  app/*/requirements.txt  .dockerignore
  scripts/            all seven: deploy, render, build-images, verify,
                      generate-load, failure-drill, evidence
  terraform/shared/cloud-init.yaml   one bootstrap, all three clouds
  terraform/platform/provider.tf  variables.tf  outputs.tf
  terraform/infra-*/provider.tf   variables.tf
  docs/CLOUD-CONTRACT.md   TROUBLESHOOTING.md
  docs/INGRESS-AND-TLS-PRIMER.md     Ingress and cert-manager from scratch
  docs/CRITICAL-EVALUATION-TEMPLATE.md
  manifests/00-namespace.yaml
  RUNBOOK.md

WORKED EXAMPLE
  manifests/03-catalog.yaml          -> the pattern for 02, 04, 05
  app/catalog/Dockerfile             -> the pattern for orders, frontend
  monitoring/servicemonitors.yaml    -> how operator discovery works
  terraform/infra-*/outputs.tf       -> names given, values yours
  docs/examples/EXAMPLE-ingress.yaml -> the shape of an Ingress. Plain HTTP,
                                        no host match, no TLS. Apply it,
                                        break it, DELETE it. Not submitted.

TO WRITE
```

```
  Task 1   terraform/infra-<your cloud>/main.tf
           terraform/infra-<your cloud>/outputs.tf   (fill in the values)
  Task 2   terraform/platform/main.tf
  Task 3   app/orders/Dockerfile
           app/frontend/Dockerfile
  Task 4   manifests/01-config.yaml
           manifests/02-redis.yaml
           manifests/04-orders.yaml
           manifests/05-frontend.yaml
  Task 5   manifests/06-tls.yaml
           manifests/07-ingress.yaml
  Task 6   monitoring/alerts.yaml            at least 5 rules, 3 categories
           one business metric of your own in app/orders or app/frontend
           (the brief wants two business metrics; the supplied orders
            service has some already, but at least one must be yours)
  Task 7   monitoring/grafana-dashboard.json   (exported from Grafana;
                                                see monitoring/README-
dashboard.md)
  Task 8   your critical evaluation report
  Task 9   nothing to write — but everything above is fair game
```

Run `./scripts/check-progress.sh` at any point to see what is still unfinished.

Why these files and not others

The split is not arbitrary. Each withheld file is the one place a criterion can actually be assessed.

You are given all the application source. Writing Flask is not assessed here, and you did it in Lab 9.3. What is assessed is deploying, securing, observing and reasoning about it. Nothing in Criteria 1–9 is measured by whether you can write `@app.get("/healthz")`.

You are given every script. `verify.sh` in particular is a specification, not a crutch — read it early and you will know exactly what "finished" means, including the object names your manifests have to use.

You are given one example of each pattern. The catalog Deployment shows you the YAML shape; writing the other three is where you make the decisions that differ between them. The ServiceMonitor shows you how operator discovery works; writing the alert rules is where PromQL judgement lives.

You are given both the contract and the bootstrap. `outputs.tf` fixes the four names so your work plugs into everything above it, and `shared/cloud-init.yaml` means nobody loses a weekend to disk device naming. What remains in `main.tf` is the network and security design, which is the whole of Criterion 1.

Task 5 gets a primer and an example, because it was not taught. The Ingress resource and cert-manager did not appear in the labs, so `docs/INGRESS-AND-TLS-PRIMER.md` introduces both from first principles and `docs/examples/EXAMPLE-ingress.yaml` is a working Ingress you can apply and poke at. Neither gives you the hostname routing, the TLS block, the redirect or the certificate chain — those are the 13 marks.

You are given `provider.tf` everywhere, because version pinning and credential handling are setup, not design. Pinning is still worth a mark — just not a mark for typing it out.

Marks, so you can budget your time

Task	What you write	Marks
1	`infra-<cloud>/main.tf`, `outputs.tf` values	12
2	`platform/main.tf`	10
3	two Dockerfiles	12
4	four manifests	12
5	`06-tls.yaml`, `07-ingress.yaml`	13
6	`alerts.yaml`, one custom metric	14
7	the exported dashboard	7
8	the evaluation report	10
9	code defense	10

Note the shape of that table. **Tasks 5 and 6 are worth 27 marks between them** and are the two hardest things here. Tasks 3 and 4 are worth 24 and are largely pattern-following. Do not spend your last week polishing Dockerfiles.

Note also that **Tasks 8 and 9 are 20 marks for explaining rather than building**. A deployment that works but that you cannot account for is a 60-mark submission at best. Keep notes on *why* as you go; you will not reconstruct your reasoning three weeks later.

The questions behind each file

Every skeleton ends with "how to check". These are the questions behind them — the ones the code defense will actually ask.

Task 1. Why not the default network? What is on your firewall and what is deliberately absent from it? How much of this moves to another cloud, and what does not?

Task 2. Why two Terraform stages rather than one? What breaks if you merge them, and why does `depends_on` not fix it?

Task 3. Why is `requirements.txt` copied before the source? Why one gunicorn worker? What is in your image that should not be?

Task 4. Why does Redis use a different update strategy from the other three? Why does the orders readiness probe not check Redis? What does a Kubernetes Secret actually protect against?

Task 5. Why not just expose the frontend with a NodePort or a LoadBalancer? Why three objects in the certificate chain and not one? Where does a CA ClusterIssuer look for its signing key? Why is the browser warning the correct outcome, and what exactly is it warning about? Why is the assessed certificate not from a public CA?

Task 6. Why `rate()` and not the raw counter? Why a ratio and not a count? Why a percentile and not a mean? What does `for:` do, and what happens without it?

Task 7. Why is RED at the top and USE below? Which panel would you look at first at 3am, and why is it where it is?

Two rules that override everything else

Never commit a secret. A live credential, private key, kubeconfig or real password anywhere in your submitted archive caps Criterion 4 at *Not achieved*, however good the rest is. Check before you submit:

```
grep -rIn "password\|BEGIN.*PRIVATE KEY\|client_secret\|access_key" . \
  --exclude-dir=.git --exclude-dir=.terraform
```

`.gitignore` is supplied and already excludes state files, `.terraform/` and `k8s-key*`. Do not weaken it.

Build everything with Terraform. Resources in Tasks 1 and 2 created by hand in a web console earn nothing in those criteria, even if the resulting system works perfectly. If you click something while debugging, destroy it and express it in code.